# Bryan Hadden – Forward Deloyed Engineer (Node/Typescript, Python, MCP, RAG)

My GitHub | 678-459-8384 | bryanchrishadden@gmail.com
Portfolio | SiteOne Salesforce Plus NCR Chick-fil-A RoomsToGo | My Website

## Summary of Experience

Full-stack engineer with 10+ years of experience building SPAs, HTTP APIs, and the data layers behind them in Node.js and Python. I've designed and owned microservices, including GraphQL Federation gateways, gRPC/Protobuf services, and FastAPI services with Pydantic validation. I've modeled relational data and SQL Server schemas for demanding financial systems, and led CI/CD pipeline buildouts with zero-downtime deploys and graceful service shutdowns

I led NCR's 50,000+ page documentation initiative for clients like Starbucks and Walmart. I also built a RAG-based PR review assistant: GitHub webhooks trigger an AWS Lambda function that uses Tree-sitter to split code into functions and classes, retrieves related context, and generates review feedback. It cut developer review times from days to hours. I use AI coding tools like Cursor, Claude, and Warp daily and I own the quality of what ships.

Outside work, I built a React Native app that predicts inventory demand using recurrent neural networks. The MVP was acquired, and a team of engineers is now scaling it, so I know how to take an idea from zero to something people pay for. I'Organizations face the problem I've been solving for years: valuable work scattered across documents, code, and data that's hard to find and connect. I can build the search, retrieval, and data pipelines that bring the right context to the right person and ship them end to end.

### National Christian Foundation | Full Stack Developer November 2025 - Present

**React, TypeScript, .NET, ASP.NET Core, C#, Entity Framework Core, SQL Server, Restful APIs, Payments, Full Stack Development**

- I own end-to-end development of the back-office and customer-facing React platforms that manage billions in giver funds, grants, and disbursements. I architect reusable component systems and complex state management with TanStack Query and Context to handle multi-step grant and disbursement workflows. By replacing legacy system processes, we're cutting disbursement processing from many weeks to days/hours while keeping critical production incidents low and customer trust high.
- Each quarter I spearhead dozens of fast, resilient features from the design validation stage to testing from business partners: PostgreSQL gRPC schemas emphasize business requirements, queries with Prisma ORM for giver, fund, and transaction records, with user data fetched data through TanStack Query with exponential backoff retries.
- I've architected the system so that data-heavy views are optimized with React.Memo, useMemo, and useCallback. As a result, financial records stay consistent, the app recovers from API failures without staff noticing because contracts match end to end, and key screens load 2-3 faster than the legacy app.
- I'm currently building an MVP MCP server in Python that gives AI agents a first-pass code review on GitHub PRs. It uses the MCP SDK, the Anthropic API (Claude), and the GitHub CLI and GraphQL API. The server parses unified diffs into hunks, maps Claude's findings to exact diff lines, and posts them as inline PR comments. It reviews each new head commit once, checking GitHub for an existing bot review first so restarts and reruns don't post duplicates. If a PR has unresolved review threads from its earlier reviews, it holds the PR and reviews the new commit only after those threads are resolved.

### Bryan Hadden LLC | Forward Deployed Engineer February 2023 - present

**Typescript, NodeJS, GraphQL, Microservices, Dev Ops**

- Architected and deployed a full-stack ML platform, with TensorFlow LSTM and feedforward models behind a Flask REST API (7 prediction endpoints) and a React Native app, delivering real-time sales forecasting and inventory demand prediction at sub-second latency.
- Built an AI-assisted pull request reviewer in Python that watches GitHub PRs and local Git branches, splits diffs into hunks with line-accurate mapping, sends them to the Claude API for correctness and security review, and posts inline review comments through the GitHub CLI; exposed the pipeline as an MCP server with six tools so Claude Code can drive scan, review, and post, with guardrails that post each PR commit once and hold PRs while unresolved bot threads remain.
- Implemented Protobuf/gRPC for service-to-service communication in the checkout API, replacing untyped

JSON with schema-first contracts that generate typed clients and enforce compatibility across microservices, with binary payloads over HTTP/2 multiplexing.

- Assembled a 5-document runbook corpus for a RAG troubleshooting assistant covering two FastAPI services (Python, Redis, PostgreSQL) packaged with Docker and Helm and run on Kubernetes, with guides for EKS, AKS, and GKE. It answers questions like "why is my rollout stalled" or "why does my Service have no endpoints" with step-by-step diagnosis and fixes, drawn from eight drills covering wrong image tags, bad Redis addresses, label mismatches, OOMKilled pods, PodDisruptionBudget-blocked node drains, autoscalers with no metrics, and ingress 502s.
- Built shortlink, a FastAPI and Redis URL shortener packaged as a hardened multi-stage Docker image (non-root, read-only root filesystem) and deployed to Kubernetes with a Helm chart covering Deployment, Service, ConfigMap, HPA, PodDisruptionBudget, and Ingress. Liveness and readiness are split so a Redis outage removes pods from service without restarting them, and GitHub Actions runs unit tests, chart linting, and a kind-cluster smoke test on every PR, publishing images to GHCR on tags.

**Bed Bath & Beyond** | **Lead React & React Native Developer** February 2025 - August 2025

**Typescript, NodeJS, GraphQL, Microservices, Dev Ops**

- Reduced Service to Service RCP Microservice response times by 40% via Protobuf/gRPC implementation using binary over HTTP/2 instead of JSON typed schemas
- I designed and built a federated gateway on Node.js (Apollo Gateway/Router) that unified 12 subgraph services – the query planning/response caching I implemented cut microservice response times by 40%.
- Optimized architecture configs files, env variable delivery and environment resulting in 25% improved system performance, validated with Datadog metrics
- Owned the checkout subgraph which handled cart totals, shipping and address selection, promotions and order submission, and it resolved shared Cart and Customer entities with the other subgraphs - It stored order and cart state in PostgreSQL with Redis caching for pricing and inventory lookups
- Optimized API retries via Redis fast count caching, spearheaded new api versioning strategy, architected cursor based pagination flow over old offset approach resulting in 25% less time to deliver products to PLP/PDP

**SiteOne Landscape Supply** | **Lead React/NextJS Developer** August 2023 - February 2025

**Typescript, NodeJS, Microservices, SAP Hybris/Spring Boot, Cyber Security, Restful APIs, GraphQL APIs**

- Led and mentored a team of 15, focusing on complex full stack features such as integrating React applications with microservices that I built and owned.
- I designed, built and owned a Node.js (Express) service for the checkout address flow, so customers can add new addresses and reuse saved ones. It syncs with SAP Hybris and cached results in Redis reducing checkout queue time by 30%.
- Modernized frontend review process to allow Devs & QA to review and bug fixes with the tools they already use (Slack, Office) - reduced review days by 15%.
- Implemented robust microservices architecture that improved system scalability, enabling the platform to handle increased user traffic during peak periods without compromising performance.
- Fostered collaboration between development, design, and product teams, ensuring alignment on feature requirements and enhancing the overall user experience through iterative feedback loops.

**NCR** | **Lead React/NextJS Developer** February 2021 - August 2023

**Typescript, Microservices, Dev Ops, AWS, Restful APIs, GraphQL APIs, MongoDb**

- Led a team of 10 engineers in the hands-on development of a NextJS application, managing client and frontend state, backend state, controllers, and microservices to ensure seamless functionality and performance.
- I spearheaded the platform that maintained more than 50,000 pages of developer documentation for clients including Walmart and Starbucks. I wrote the technical guidelines, specs, testing strategy and roadmap for it.
- I built the Next.js application end to end, including its Node.js API routes/BFF layer, controllers and microservices and an MDX content pipeline that turned Markdown and API specs into versioned, per-client reference pages
- Developed and maintained technical guidelines, specifications, testing strategies, and roadmaps for the documentation initiative, ensuring that every design detail was accurately represented and executed in the final product.
- Owned the Next.js docs platform end to end, including a Node.js content service that pulled docs from Git and a headless CMS, validated links and metadata, and published updates through incremental

static regeneration.
- Championed Agile development methodologies within the team, facilitating regular sprint reviews and retrospectives that improved collaboration and increased feature delivery speed by 30%.
- Modeled MongoDB collections for client-specific documentation, API references, and release metadata; created compound indexes on client, product version, and page slug to support fast, version-aware lookups across tens of thousands of pages.

**Freelancer** | **Sr React/NextJS Developer** December 2019 - February 2021

**Typescript, Python, Salesforce Lightning Components, Microservices, Automated Testing, Test Driven Development**

**Disney**

- Owned Python microservice with a company-wide impact, Lucas Films, Parks, Pixar etc.
- Used jQuery and Vanilla JavaScript to build out the entire suite of applications

**Chick-fil-A**

- In the first two months took ownership of and delivered the favorites React feature which makes up about 1/3rd of the current app; Launched the new CFA web app nationally & internationally.
- Collaborated with Business, UX and Product to implement efficient design patterns.

**Salesforce**

- Rebuilt Featured Episode/Original Series for Salesforce Plus
- Led team of 30; Daily mentoring, pairing & code reviews

**Porsche** | **Lead React & Python Developer** September 2018 – December 2019

**Typescript, Python, Microservices, AWS, Restful APIs**

- I built a full-stack, real-time vessel-tracking platform (Python backend, map-based React frontend) for car-carrying ships. Logistics teams in the US and Germany and crews on board used it, and it gave visibility into more than €3B of product.
- Upon replacing previous system, we achieved a 42% reduction in fixed cost across frontend/backend systems
- Designed a layered backend in which routers and controllers pulled live ship data from third-party APIs, and a service layer parsed internal emails and shipping documents and normalized them with schemas into one tracking view
- Owned app with 100+ users which included everyone from shipping/port managers to middle management and C-Suite – also owned AWS Identity and access management.

**Home Depot** | **Vanilla JavaScript/React Developer** May 2017 – September 2018

**Typescript, Microservices, Google Console, Restful APIs, MongoDB**

- Owned and maintained an internal design system with 500+ components shipped in Angular, React, and vanilla JavaScript versions with weekly optimizations and quality of life improvements for over 250 designers and 7000+ developers
- Spearheaded design system integration into a Flask-backed login app, delivering accessible (WCAG 2.1 AA) form, input, and error-state components that reduced login UI build time from 3 hours to 20 minutes, which let us deploy changes several times a day instead of several times a week
- Hardened the login flow with rate limiting (Flask-Limiter), CSRF protection, and account lockout after [5] failed attempts, reducing brute-force login attempts and aligning employee time keeping to further match internal policies around time in/time out entry and reducing overtime discrepancies
- Rebuilt state management for a real-time employee login system using NgRx store, effects, and memoized selectors, and moved 20+ components to OnPush change detection, cutting change-detection cycles by half which increased employes adoption from 10 to over 1000 in 6 months
- Replaced component-level HTTP calls with a centralized NgRx entity store, so employee dashboard widgets shared one data source instead of fetching separately, reducing page load API calls from 100 per page load to 5
- Integrated MongoDB with the Flask employee login system using PyMongo; modeled employee and time-entry records and added indexes for employee and timestamp lookups to keep attendance history and dashboard data responsive as adoption grew.

**Freelance** | **WordPress/React Developer & Designer** January 2015– May 2017

**WordPress, Laravel, PHP, CSS, HTML, Figma**

- Developed and deployed multiple WordPress sites on AWS EC2 instances, managing server configuration via SSH and implementing custom PHP components for client projects
- Leveraged React and Angular frameworks to modernize legacy interfaces within the internal development framework, supporting hundreds of designers in their workflow
- Designed user interfaces in Figma and translated them into functional WordPress sites with responsive, mobile-first designs

## Education

**Foundations of Science Engineering - MIT** *Jan - May 2017*

**Computer Science - Winston-Salem State University** May *2006 – December 2010*